

Lab 02 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — Volledige namen en korte namen

Antwoord.

Schrijfwijze	Volledige naam	Podman
nginx	docker.io/library/nginx:latest	gekende alias → opgelost zonder vraag, melding <code>Resolved "nginx" as an alias</code>
bitnami/nginx	docker.io/bitnami/nginx:latest	geen alias → zoekt in <code>unqualified-search-registries</code> ; één registry op Ubuntu (<code>docker.io</code>), dus opgelost; meerdere op Fedora, dus een vraag
registry.mijnbedrijf.be:5000/basis/nginx:1.25	ongewijzigd	volledige naam: geen resolutie

De regel: als het eerste deel (vóór de eerste `/`) een **punt** of een **dubbelpunt** (poort) bevat, of `localhost` is, is het een registry. Anders is het een namespace op de standaardregistry.

Waarom. `mijnbedrijf.be` kan geen gebruikersnaam op Docker Hub zijn (punten zijn er verboden), en een poort heeft alleen zin voor een server. Docker past die regel toe en vult dan stilzwijgend aan; Podman past dezelfde regel toe maar weigert blindelings aan te vullen, want `nginx` op `docker.io` en `nginx` op `registry.intern` kunnen twee verschillende images zijn.

Nuance. `bitnami/nginx` is **geen** officiële image (namespace `bitnami`, niet `library`), ondanks de naam. En een image die je zonder registry bouwt, wordt `localhost/...`: een volledige naam, met `localhost` als fictieve "registry".

Voorbeeld.

```
podman pull nginx 2>&1 | head -2          # Resolved "nginx" as an alias ...
docker.io/library/nginx:latest
podman image inspect --format '{{.RepoTags}}' nginx
podman build -t api:1.0 . && podman images | grep api    # localhost/api 1.0
```

Vraag 2 — Zelfde tag, andere inhoud

Antwoord. De tag `2.3` is intussen **verplaatst**: iemand heeft een image opnieuw gebouwd en gepusht onder dezelfde naam. A behoudt de oude image (geen pull), B kreeg de nieuwe. Je bewijst het door de digests te vergelijken; je vermijdt het door nooit een gepubliceerde tag te hergebruiken en op digest uit te rollen.

Waarom. Een tag is een muteerbare pointer aan registryzijde. De `pull` vergelijkt de digest op afstand met de lokale en downloadt alleen als ze verschillen. Niets waarschuwt dat een tag

bewogen is.

Nuance. De verplaatsing kan onbedoeld zijn: een pipeline die bij elke run op de releasebranch `api:2.3` pusht, of een `latest`. Ook de basisimage kan bewogen zijn zonder dat je Dockerfile verandert (`FROM eclipse-temurin:21-jre`): "dezelfde code" opnieuw bouwen geeft een andere image.

Voorbeeld.

```
# op A en op B:
podman image inspect --format '{{.Digest}}' mijnapp/api:2.3
# verschillende digests -> de tag is bewogen. Correcte uitrol:
podman pull registry.intern/mijnapp/api@sha256:9d0d1f1e...
```

Vraag 3 — 62 GB getoond, schijf intact

Antwoord. Nee. `SIZE` geeft de **virtuele** grootte van elke image, gedeelde lagen inbegrepen: gemeenschappelijke lagen (JRE, Alpine, Debian) worden in elke image meegeteld maar één keer opgeslagen. `podman system df` geeft het echte gebruik. De bestanden staan in `~/.local/share/containers/storage` — in de virtuele schijf van de WSL-distributie (`ext4.vhdx`), niet in een Windows-map.

Waarom. Het stuurprogramma `overlay` slaat elke laag één keer op, geïdentificeerd op inhoud, en images zijn maar lijsten van lagen. Twaalf Spring Boot-images op dezelfde JRE delen zijn 180 MB.

Nuance. De `vhdx` van WSL **groeit** automatisch maar **krimpt** niet vanzelf wanneer je images verwijdt: de ruimte komt vrij aan Linux-zijde, niet aan Windows-zijde, zolang je de schijf niet gecompacteerd hebt (`wsl --shutdown` en dan `Optimize-VHD` of `diskpart`). Een frequente verrassing op een Windows-werkpost.

Voorbeeld.

```
podman system df                # echte SIZE en RECLAIMABLE
podman system df -v | head      # kolom SHARED SIZE per image
podman info --format '{{.Store.GraphRoot}}' # /home/<jij>/local/share/containers/storage
```

Vraag 4 — De `rm` die niets verwijdt

Antwoord. Hij heeft ongelijk. `COPY` maakt een laag aan die `credentials.json` **bevat**. De `RUN ... rm` maakt een latere laag aan met een *whiteout* die het bestand verbergt. De uiteindelijke image bevat beide lagen: het bestand is aanwezig, alleen onzichtbaar vanuit een container.

Waarom. Lagen zijn onveranderlijk en additief. Een laag kan geen bestand uit een vorige laag wegnemen; ze kan het alleen verbergen. Wie de image heeft, kan ze bewaren met `podman save`, de laag van de `COPY` uitpakken en het bestand lezen.

Nuance. Zelfs in **één enkele** `RUN` (`COPY` en dan `rm` in dezelfde instructie) blijft de `COPY` een aparte instructie met een eigen laag. Alleen een *build secret* (`RUN --mount=type=secret`) of een multi-stage build (lab 05) vermijdt de aanwezigheid van het bestand in de uiteindelijke image.

Voorbeeld.

```
podman save --format oci-archive -o img.tar mijn-image:1.0
mkdir x && tar -xf img.tar -C x
for b in x/blobs/sha256/*; do tar -tf "$b" 2>/dev/null | grep -q credentials.json && echo
"aanwezig in $b"; done
```

Vraag 5 — 310 MB, 61 MB overgedragen

Antwoord. De registry heeft de ongewijzigde lagen al (JRE, systeem, afhankelijkheden); de `push` draagt alleen de nieuwe lagen over — de JAR en wat erna komt — 61 MB dus. Als alles telkens opnieuw overgedragen werd, verandert de eerste laag van de image bij elke build: een `COPY . .` te vroeg, of een instructie met variabele inhoud (datum, versie) vóór de zware lagen.

Waarom. Elke laag heeft een digest. Voor het verzenden van een blob vraagt de client de registry of ze hem heeft (`HEAD /v2/<repo>/blobs/<digest>`). Podman toont dat minder expliciet dan Docker (geen `Layer already exists`), maar de overdracht is onmiddellijk.

Nuance. Het delen tussen *repositories* van dezelfde registry hangt af van de implementatie (Harbor en Docker Registry doen het via *cross-repository mount*). En een "ongewijzigde" laag moet bit voor bit identiek zijn: een `RUN apt-get update` zonder vastgepinde versie produceert bij elke build een andere laag.

Voorbeeld.

```
podman push --tls-verify=false localhost:5000/basis/demo:1.1 # onmiddellijk: blobs al
aanwezig
podman history mijn-api:2.0 --format 'table {{.Size}}\t{{.CreatedBy}}' # de laag die verandert
opsporen
```

Vraag 6 — Twee tags, één image, en een spook

Antwoord. Twee verschillende images: `f3a1b9c02d11` (met de tags `2.0` en `1.9`) en `8b2c74e91a03` (zonder naam). De regel `<none>` is de *dangling* image: een tag (`2.0` , waarschijnlijk) is opnieuw gebouwd en verplaatst, de oude image verloor haar naam. `podman rmi api:1.9` verwijdert **alleen de tag** (`Untagged: localhost/api:1.9`); de data blijft, verwezen door `2.0` . `localhost/` is de fictieve registry van elke image die zonder registrynaam gebouwd of getagd is.

Waarom. Een `IMAGE ID` is de digest van de imageconfiguratie; twee regels met dezelfde ID zijn twee namen voor één inhoud. De data verdwijnt pas met de laatste naam.

Nuance. Verwar *dangling* (`<none>:<none>` , zonder enige tag) niet met *unused* (met tag, maar zonder container). Een `podman image prune` zonder `-a` verwijdert alleen de eerste.

Voorbeeld.

```
podman rmi api:1.9 # Untagged: localhost/api:1.9 (geen Deleted)
podman images --filter dangling=true -q # 8b2c74e91a03
podman rmi 8b2c74e91a03 # Deleted: ... deze keer verdwijnt de data
```

Vraag 7 — `exec format error`

Antwoord. De image is gebouwd voor `linux/arm64` (Apple Silicon) en de server is `linux/amd64`: de kernel kan het binary niet uitvoeren. Onmiddellijke oplossing: opnieuw bouwen met `--platform linux/amd64` (QEMU-emulatie, traag maar werkend). Duurzame oplossing: de images laten bouwen door de **CI** op `amd64`-agents, of multi-architectuur-images produceren (`podman manifest`).

Waarom. Een multi-arch-tag is een manifest list; bij de `build` produceert de engine een manifest voor de architectuur van de bouwmachine. Bij de `pull` kiest de server de `amd64`-ingang... die niet bestaat, dus krijgt hij `arm64`.

Nuance. *Officiële* images zijn bijna allemaal multi-arch, wat het probleem verbergt tot de eerste eigen build. En de fout kan vroeger opduiken: `podman run` op de Mac van een `amd64`-image werkt (emulatie), maar 5 tot 10 keer trager.

Voorbeeld.

```
podman image inspect --format '{{.Architecture}}' registry.intern/api:1.4 # arm64
podman build --platform linux/amd64 -t registry.intern/api:1.4 .
```

Vraag 8 — `save` tegenover `export`, en het OCI-formaat

Antwoord. `save` exporteert een **image**: lagen, manifest, configuratie, tags. `export` exporteert het bestandssysteem van een **container**, platgeslagen tot één boom, zonder metadata. Om een Spring Boot-image naar een geïsoleerde site te vervoeren is `save` de juiste keuze. Met `export` verlies je `ENTRYPOINT`, `CMD`, `ENV`, `EXPOSE`, `USER`, `WORKDIR` — de geïmporteerde image weet niet meer hoe te starten — en ook de lagen (geen delen of cache meer). `--format oci-archive` produceert dezelfde image in de OCI-standaardindeling (`blobs/sha256/`, `index.json`) in plaats van het historische formaat van Docker.

Waarom. `export` ziet alleen het resultaat van de assemblage van de lagen, zoals een `tar` genomen vanuit de container. De configuratie leeft in de image, niet in het bestandssysteem.

Nuance. `export` heeft een legitiem gebruik: een bestandssysteem ophalen voor analyse, of een "platte" image maken vanuit een handmatig geconfigureerde container (slechte praktijk, maar gedocumenteerd). Het formaat `docker-archive` blijft het meest voorkomende; `oci-archive` gebruik je als de ontvanger geen Docker is (Kubernetes via `ctr`, skopeo...).

Voorbeeld.

```
podman save --format oci-archive -o api.tar mijn-api:1.0
podman load -i api.tar # Loaded image: localhost/mijn-api:1.0
podman export c1 | podman import - plat:1 # Config.Cmd = null
```

Vraag 9 — De tweede `pull`

Antwoord. De engine heeft de registry om het **manifest** van de tag gevraagd, de digest ervan vergeleken met die van de lokale image, vastgesteld dat ze identiek zijn, en verder niets gedownload. Het manifest is enkele kilobytes: de netwerkkost is die van een of twee HTTP-verzoeken.

Waarom. Elke laag en het manifest worden op inhoud geadresseerd; de client weet precies wat hij bezit. Een pull is altijd differentieel, en in het uiterste geval leeg.

Nuance. Podman zegt niet "Image is up to date": het toont gewoon de identificatie van de image. En "enkele seconden" veronderstelt een nabije registry; tegenover Docker Hub kan het verzoek meerdere seconden latentie hebben, zonder iets te downloaden. Ten slotte telt dat verzoek mee in het quotum van Docker Hub (vraag 10).

Voorbeeld.

```
time podman pull alpine      # d529dd0c... – enkele seconden netwerk, nul lagen overgedragen
```

Vraag 10 — toomanyrequests

Antwoord. Docker Hub begrenst anonieme `pull`s per IP-adres (en per account voor geauthenticeerde gebruikers). De CI-agents gaan allemaal naar buiten via hetzelfde publieke IP: het quotum wordt gedeeld door het hele bedrijf, vandaar "willekeurige" mislukkingen naargelang de belasting van het moment. De twee antwoorden: een **pull-through cache** (interne registry die Docker Hub cachet) en/of de **interne kopie** van de basisimages in de bedrijfsregistry (`skopeo copy`), met Dockerfiles die naar die registry verwijzen.

Waarom. Elke `pull` bevraagt minstens het manifest, zelfs als de image al lokaal is. Een CI die honderd keer per dag bouwt, overschrijdt het quotum snel.

Nuance. Authenticeren (`podman login docker.io`) verhoogt het quotum maar heft het niet op, en zet een persoonlijk account in de CI. De interne kopie heeft een bijkomend voordeel: je controleert *wat binnenkomt* (scan, validatie), en je hangt niet meer af van de beschikbaarheid van Docker Hub.

Voorbeeld.

```
skopeo copy docker://docker.io/library/node:22-alpine docker://registry.intern/basis/node:22-alpine
# en dan in de Dockerfile: FROM registry.intern/basis/node:22-alpine
```

Vraag 11 — HTTP tegenover HTTPS

Antwoord. Docker behandelt `localhost` (en `127.0.0.0/8`) standaard als een *insecure* registry: het aanvaardt HTTP zonder iets te zeggen. Podman kent geen uitzondering: elke registry moet een geldig TLS-certificaat voorleggen. Twee manieren om erdoor te komen: `--tls-verify=false` op het commando, of een ingang `[[registry]] location = "localhost:5000" insecure = true` in `registries.conf` . De tweede mag nooit in een geversioneerd bestand of op een server belanden: ze schakelt de verificatie uit voor **alle** gebruik van die registry, stilzwijgend.

Waarom. Een registry zonder TLS kan door eender wie op het netwerkpad nagebootst worden (*man in the middle*) en een besmette image terugsturen. Op `localhost` is het risico klein; Podman heeft liever dat je het zegt dan dat het het veronderstelt.

Nuance. `--tls-verify=false` schakelt ook de **certificaat**verificatie uit op een zelfondertekende HTTPS-registry — de goede praktijk is eerder het certificaat van de interne autoriteit te installeren in `/etc/containers/certs.d/<registry>/ca.crt` .

Voorbeeld.

```
podman push --tls-verify=false localhost:5000/basis/demo:1.0      # expliciet, zichtbaar in de
geschiedenis
# OF, alleen voor een ontwikkelwerkpost:
printf '[[registry]]\nlocation = "localhost:5000"\ninsecure = true\n' >>
~/.config/containers/registries.conf
```

Vraag 12 — image is in use by a container

Antwoord. Er bestaat nog een container (aangemaakt vanuit die image, zelfs gestopt); de image is zijn basislaag. De engine weigert omdat de image verwijderen die container zou breken. Netjes: die container oplijsten (`podman ps -a --filter ancestor=...`), hem verwijderen als zijn toestand niet meer dient, en dan `rmi`. `podman rmi -f` verwijdert de tag en de image **én** de container die ervan afhangt, zonder vragen: je verliest de logs, de schrijflaag en de mogelijkheid om de container te inspecteren — om tien seconden te winnen.

Waarom. Een container is image + schrijflaag. Zonder de image betekent de schrijflaag niets meer.

Nuance. De melding van Podman spreekt over "externe containers": die welke door Buildah of een andere tool met dezelfde opslag zijn aangemaakt, onzichtbaar in `podman ps`. `podman ps -a --external` toont ze. Docker weigert ook, maar zijn melding noemt de identificatie van de container voluit.

Voorbeeld.

```
podman ps -a --filter ancestor=mijn-api:1.0 --format '{{.Names}} {{.Status}}'
podman logs <container> > incident.log      # we bewaren wat bewaard moet worden
podman rm <container> && podman rmi mijn-api:1.0
```

Vraag 13 — De lagen van 0B

Antwoord. Het zijn **metadata**-instructies — `ENV`, `CMD`, `ENTRYPOINT`, `EXPOSE`, `LABEL`, `USER`, `WORKDIR` — die de configuratie van de image wijzigen zonder het bestandssysteem aan te raken. Ze verschijnen in de geschiedenis omdat elke instructie een spoor nalaat, ook een leeg. De laag van 180 MB (een `RUN apt-get install`, een `COPY` van een JRE) is de enige die telt: `podman history` wijst de te optimaliseren regel aan.

Waarom. Een image is een reeks instructies met, voor sommige, een bijbehorende blob aan bestanden. Het gewicht komt alleen van de blobs.

Nuance. Een niet-lege laag kan een verwijdering verbergen: `RUN rm -rf /var/lib/apt/lists/*` in een aparte `RUN` weegt bijna 0B maar wint niets terug. `history` toont de kost van elke stap; `podman image tree` toont daarbovenop welke lagen van de basisimage komen.

Voorbeeld.

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}'  # 12 regels van 0B,
één van 50.7MB
podman image tree nginx:alpine
```

Vraag 14 — Drie tagstrategieën

Antwoord. (a) overschreven `latest`: **rollback onmogelijk** (de oude image heeft geen naam meer) en diagnose onmogelijk (onmogelijk te weten welke versie draaide). (b) `1.4.2`: onmiddellijke rollback naar `1.4.1`; correcte diagnose als de tag onveranderlijk is, maar twee builds van `1.4.2` (snelle fix) kunnen naast elkaar bestaan zonder dat men ze onderscheidt. (c) `1.4.2-b318-a9f3c21`: elke image is uniek, men gaat terug tot de exacte commit en build; rollback naar eender welke eerdere build. De prijs is een onleesbare naam en een te beheren retentie.

Waarom. Uitrol en diagnose hebben een **één-op-één**-verband nodig tussen een naam en een inhoud. Alleen (c) garandeert dat door constructie; (b) garandeert het door discipline; (a) verbiedt het.

Nuance. De drie bestaan vaak samen: de CI pusht (c); een tag (b) wordt *toegevoegd* aan dezelfde image wanneer ze gevalideerd is; `latest` bestaat alleen voor het gemak van ontwikkelaars, nooit in een uitrolmanifest. En de uitrol zelf pint de digest vast.

Voorbeeld.

```
podman tag registry.intern/api:1.4.2-b318-a9f3c21 registry.intern/api:1.4.2 # zelfde image,
tweede naam
podman image inspect --format '{{.Digest}}' registry.intern/api:1.4.2 # wat werkelijk
uitgerold wordt
```